

28 **【命名】**

29

30 保留字(例如if/else/const...)不可以當代數名稱

31

32 變數名稱中間不能空格 可用底線代替(蛇是)或用大寫(駝峰式)

33 示範：

34 my_student_score

35 myStudentScore

36 駝峰式常用於 C/C++/Java/JS

37 蛇式常用於 Python/Ruby

38

39 變數命名第一個字不能是數字

40 注意大小寫有別

```
42    【var宣告】
43    var a = 1;
44    console.log(a);
45    結果就是1
46
47    a;
48    console.log(a)
49    結果就是not defined沒有定議
50
51    var a;
52    console.log(a)
53    結果就是undefined未定議
```

```
55  const a = ["x", "y", "z"];
56  console.log(a);
57  const(常數宣告)不能re-assign(重新指定)
58  意思是[]裡面的xyz可以單一換掉比如把y換掉 甚至可以刪掉xyz
59  但不能把整個[]換掉
60  即便給新的{xyz}也不行
61
62  const friends = ["a", "b", "c"]
63  console.log(friends)
64  friend[0] = "x"
65  因為a的位子是0 b的位子是1 c的位子是2
66  console.log(friends)
67  結果:
68  ["a", "b", "c"]
69  ["x", "b", "c"]
```

```
Windows PowerShell
著作權 (C) Microsoft Corporation。保留擁有權利。

安裝最新的 PowerShell 以取得新功能和改進功能！https://aka.ms/PSWindows

PS C:\Users\Galaxy> node -v
v24.14.1
PS C:\Users\Galaxy> |
```

node -v就是確認裝好
node了沒?裝了幾號版本?

```
問題 輸出 終端機 ... + v powershell
PS C:\Users\Galaxy\Desktop\Demo> node aaa.js
123
PS C:\Users\Galaxy\Desktop\Demo> |
```

```
72 let age = 10;
73 如果我知道上面這行年齡之後不會改 就用const
74 如果會改就用let(變數宣告)然後盡量不要用var(也是變數宣告)
75 if (age >= 18) {
76     | console.log("已成年");
77 }
78 if (age <= 17) {
79     | console.log("未成年");
80 }
81 上方這個未成年設定更快的做法是:
82 if (age >= 18) {
83     | console.log("已成年");
84 } else {
85     | console.log("未成年")
86 }
87
```

```
88  【if】
89  判斷偶數：
90  const n = 17
91  if ( n %2 == 0) {
92  |    console.log("偶數");
93  } else {
94  |    console.log("奇數")
95  }
96  ==是等於
97  判斷奇數：
98  const n = -17
99  if ( n %2 != 0) {
100 |    console.log("奇數");
101 }
102 !=是不等於
103
104 但是單一個=不是等於的意思 是指定的意思
```

```
JS os.js > [y]
1  const y = 2026
2
3  if ( y % 4 == 0) {
4  |    if (y % 100 == 0) {
5  |        if (y % 400 == 0) {
6  |            console.log("閏年")
7  |        }else {
8  |            console.log("平年")
9  |        }
10 |    }else {
11 |        console.log("閏年")
12 |    }
13 }else {
14 |    console.log("平年")
15 }
```

問題 輸出 偵錯主控台 終端機 連接埠 + v powershell

```
PS C:\Users\Galaxy\Desktop\Demo> node os.js
平年
PS C:\Users\Galaxy\Desktop\Demo> █
```

快速修復檢查清單

1. 設定預設格式化工具 (最常見原因)

- 在 VS Code 按下 `Ctrl + Shift + P` (Mac: `Cmd + Shift + P`)。
- 輸入 `Preferences: Open Settings (JSON)`。
- 確保有以下設定：

json



```
"editor.defaultFormatter": "esbenp.prettier-vscode",  
"editor.formatOnSave": true
```

2. 開啟儲存時自動格式化 (Format on Save)

- 開啟設定 (`Ctrl + ,` / `Cmd + ,`)。
- 搜尋 `format on save`。
- 勾選 **Editor: Format On Save**。

```
console.log(typeof 123) // 印出 "number"  
console.log(typeof 3.14) // 印出 "number"  
console.log(typeof "hello") // 印出 "string"  
console.log(typeof true) // 印出 "boolean"  
console.log(typeof false) // 印出 "boolean"  
console.log(typeof undefined) // 印出 "undefined"  
console.log(typeof Symbol("hello")) // 印出 "symbol"  
console.log(typeof null) // 印出 "object" !!!!
```


終端機常用指令-1

Windows	MacOS / Linux	說明
cd	pwd	取得目前所在的位置 (Print Working Directory)
cd	cd	切換目錄 (Change Directory)
dir	ls	列出目前的檔案列表 (Directory. List)
dir /a	ls -a	列出含隱藏的檔案列表 (all)
dir /a	ls -al	列出含隱藏的檔案列表及詳細資訊 (all + long)

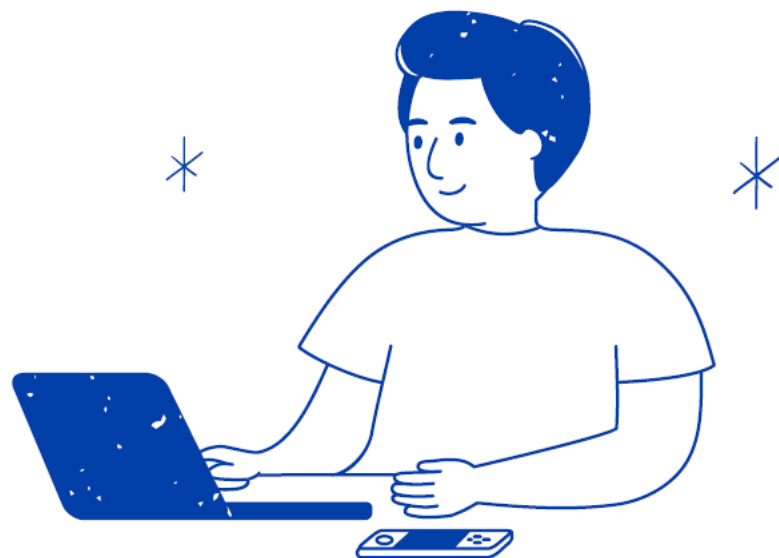
終端機常用指令-2

Windows	MacOS / Linux	說明
mkdir	mkdir	建立新的目錄 (Make Directory)
無	touch	建立檔案
copy	cp	複製檔案 (Copy)
move	mv	移動檔案 (Move)
del	rm	刪除檔案 (Delete. Remove)
cls	clear	清除畫面上的內容 (Clear Screen)

git config

- 檢查 git 設定值
\$ git config --list
- 設定 user.name
\$ git config --global user.name "Krystal"
- 設定 user.email
\$ git config --global user.email
"krystal@gmail.com"

註：有傳遞 --global 參數，只需要做一次

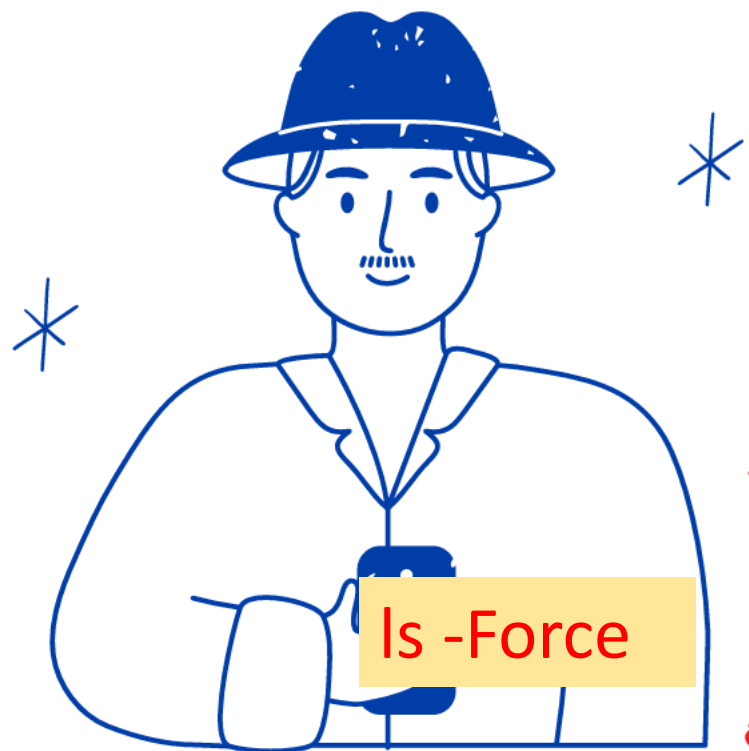


git init

用電腦終端機到
資料夾位子再輸
入code .(前面要
空格)就能開啟有
這個資料夾的
vscode

確認版本號

\$ git --version



建立資料夾，且 cd 進去

開VS的終端 \$ git init

生成 .git 資料夾

(類似於日記本、歷史紀錄)

如果沒有.git出現
就輸入
ls -Force
就會有了!

不同資料夾 git init 會發生什麼事?

若在錯的資料夾 init 怎反悔?

題外話:
以防路徑
不同
打開新視
窗再把資
料夾丟進
來

commit 三區



git add <檔案名>



將檔案變動加入暫存區

\$ git add <檔案名>

\$ git add <資料夾名>/

\$ git add .

在 git add 後又修改同檔案的內容會怎樣？

git add 後我反悔了怎辦？

```
Windows PowerShell
著作權 (C) Microsoft Corporation。保留擁有權利。

安裝最新的 PowerShell 以取得新功能和改進功能！https://aka.ms/PSWindows

PS C:\Users\Galaxy> cd desktop
PS C:\Users\Galaxy\Desktop> cd git
PS C:\Users\Galaxy\Desktop\git> code
PS C:\Users\Galaxy\Desktop\git> |
```

練習題

```
PS C:\Users\Galaxy\Desktop\git> git add aaa.js
PS C:\Users\Galaxy\Desktop\git> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   aaa.js

PS C:\Users\Galaxy\Desktop\git> git add qqqq/ppp.md
PS C:\Users\Galaxy\Desktop\git> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   aaa.js
    new file:   qqqq/ppp.md

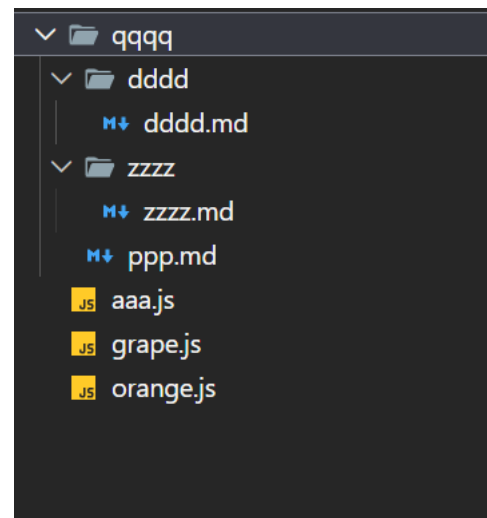
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    grape.js
    orange.js
```

```
PS C:\Users\Galaxy\Desktop\git> git add .
PS C:\Users\Galaxy\Desktop\git> git status
On branch master
```

No commits yet

```
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   aaa.js
    new file:   grape.js
    new file:   orange.js
    new file:   qqqq/dddd/dddd.md
    new file:   qqqq/ppp.md
    new file:   qqqq/zzzz/zzzz.md
```

```
PS C:\Users\Galaxy\Desktop\git> git commit -m '第一次commit'
[master (root-commit) bfe5c0c] 第一次commit
6 files changed, 1 insertion(+)
create mode 100644 aaa.js
create mode 100644 grape.js
create mode 100644 orange.js
create mode 100644 qqqq/dddd/dddd.md
create mode 100644 qqqq/ppp.md
create mode 100644 qqqq/zzzz/zzzz.md
PS C:\Users\Galaxy\Desktop\git> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\Galaxy\Desktop\git> |
```



如果已經commit過，需要修改檔案，
只要重新add再commit就好

可以一口氣add多個檔案
例如git add aaa.txt bbb.txt ccc.txt
檔案名中間空格就好
接著commit 會三個檔案一起存

Git 三區基本指令

指令	說明
git status	查詢現在這個資料夾裡檔案們的狀況
git add <檔名>	將檔案（們）從工作目錄加進暫存區（購物車）
git restore --staged <檔名>	將已經加入暫存區的檔案移除回工作目錄
git commit -m "<異動訊息>"	將暫存區的所有內容存檔至本地的 git 紀錄裡
git commit --amend -m "Hi"	修改最新一次的 commit 訊息（SHA 會變）
git blame <檔名>	看每一行程式碼是誰寫的（抓戰犯）

為什麼要二段式？



這個動作會從工作區跳過暫存區
直接到本地

\$ git commit -a -m "update content"
(-a → add)

註：只存在於已經存在的檔案異動才有用

暫存區

這個動作會從工作區跳過暫存區
直接到本地儲存區

Git 查看紀錄 - 1

指令	說明
<code>git log</code>	查看 Git 的 commit 歷史紀錄，由新到舊排列（越上面越新）
<code>git log --oneline</code>	單行顯示 commit
<code>git log -n <數量></code>	限制顯示幾個 commit (number)
<code>git log --graph</code>	看包含 branch 的走向（類似 git graph 套件結果）
<code>git log -p</code>	顯示完整修改內容 (patch)

=和後面的" 人名" 之間不可以有空格
然後人名會是灰色的 這是正常的

Git 查看紀錄 - 2

指令	說明
<code>git log --oneline --author="krystal"</code>	看特定人的 commit(多人用 \ 分隔)
<code>git log --oneline --grep="hello"</code>	看特定字的 commit
<code>git log --oneline --since="5pm" --until="6pm"</code>	特定時間區間的 commit
<code>\$ git log --oneline --since="9am" --until="12am" --after="2026-03-11"</code>	從 2026/3/11 後，每天早上 9-12 的 commit

git config 設定縮寫



```
$ git log --oneline --graph
```

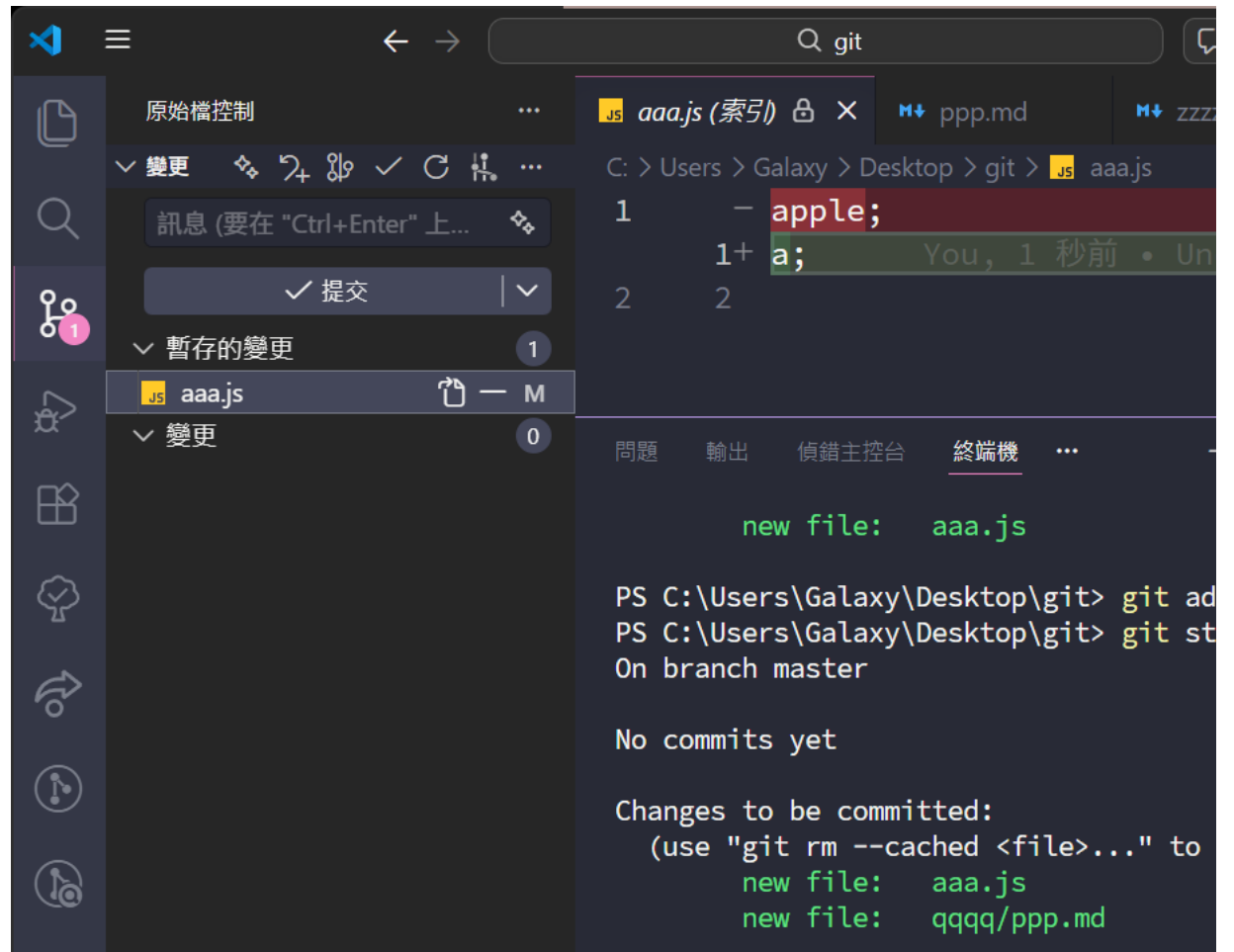
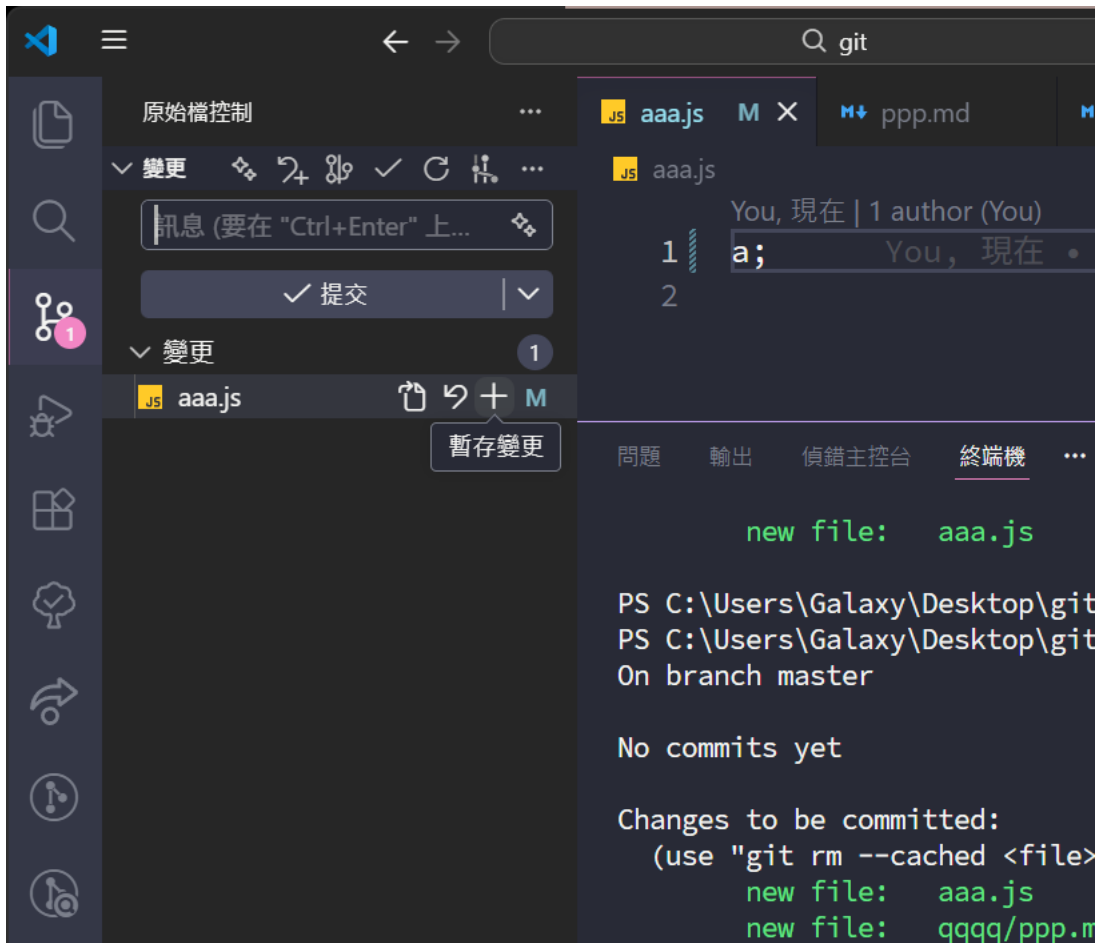
可以用 git config 設定縮寫

```
$ git config --global alias.l "log --oneline -  
-graph"
```

設定後以後可以簡化成

```
$ git l
```

可以自訂更多縮寫



The screenshot shows the VS Code GitLens interface. On the left, the 'INSPECT: COMMIT DETAILS' panel for commit 6c67588 by user 'krystal' is visible. It shows the commit message 'style: add text to the cat.html' and a list of changed files, with 'cat.html' circled in red and labeled '按' (press). Below this, the 'FILE HISTORY' and 'VISUAL FILE HISTORY' sections are shown. On the right, the 'cat.html' file is open, showing a diff with the change '可愛貓貓' (Kawaii Neko Neko) added. At the bottom, a terminal window shows the execution of 'git log' commands to find the commit SHA. The output shows the commit 6c67588 with the message 'style: add text to the cat.html'. A red arrow points from the '6c67588' SHA in the terminal to the '按' label, and another red arrow points from the '按' label to the 'cat.html' file in the file history.

GitLens INSPECT

INSPECT: COMMIT DETAILS

6c67588

krystal

style: add text to the cat.html

Learn About Autolinks

Show Branches & Tags

1 FILE CHANGED

cat.html

LINE HISTORY

FILE HISTORY cat.html

style: add text to the cat.html krystal, 上週 M

chore: add cat krystal, 上週 A

VISUAL FILE HISTORY cat.html

ursi. > cat.html Up to 3mo ago

2026/3/11 2026/3/15 2026/3/19 2026/3/23 2026/3/27

krystal

SEARCH & COMPARE

cat.html (fe1eb95) ↔ cat.html (6c67588)

D: > 下載 > git-dojo > git-dojo > 03-two-branches > cat.html

1 -

1+ 可愛貓貓 krystal, 上週 • style: add text to th

問題 輸出 偵錯主控台 終端機

PS D:\下載\git-dojo\git-dojo\03-two-branches>

PS D:\下載\git-dojo\git-dojo\03-two-branches>

PS D:\下載\git-dojo\git-dojo\03-two-branches>

PS D:\下載\git-dojo\git-dojo\03-two-branches> git log --oneline --grep

= "cat.html" PS D:\下載\git-dojo\git-dojo\03-two-branches> git log

--oneline --grep="cat.html" PS D:\下載\git-dojo\git-dojo\03-two-bran

ches> git log --oneline PS D:\下載\git-dojo\git-dojo\03-two-branches>

PS D:\下載\git-dojo\git-dojo\03-two-branches> git log --oneline --grep

= "cat.html"

6c67588 style: add text to the cat.html

PS D:\下載\git-dojo\git-dojo\03-two-branches>

按 ctrl + 這個 SHA 值

sha值就是那串數字 可以按喔

git branch 基本指令 - 1

指令

`git branch`

`git branch <分支名>`

`git branch -d <分支名>`

`git branch -D <分支名>`

說明

查看目前（本機）有哪些 branches

在現在位置，建立名為 <分支名稱> 的分支

刪除名為 <分支名稱> 的分支 (delete)

強制刪除 branch

git branch 基本指令 - 2

指令	說明
git branch -m <新分支名>	重命名目前的 branch
git branch -M <新分支名>	強制重新命名 (force rename) branch
git branch -a	查看所有 branch (包含遠端) (all)
git branch <分支名稱> <commit>	在指定的 commit (SHA) 位置建立一個新的 branch
	註：git branch new-feature HEAD 縮寫 git branch new-feature

切換 branch 基本指令 - 1

指令	說明
git switch <分支名>	將 HEAD 移到指定的 branch
git switch -c <分支名>	建立新 branch 並 <u>切換過去</u> (create)
git switch -	切換回 上一個 branch
git switch --detach <commit>	切換到指定 commit (進入 detached HEAD)

Q1: 我切換到一個沒有 branch 的 commit 會發生什麼事?

Q2: 我在這個 commit 上面再多做一個 commit 會發生什麼事?

切換 branch 基本指令 - 2

指令	說明
<code>git checkout <分支名></code>	將 HEAD 移到指定 branch
<code>git checkout -b <分支名></code>	建立 branch 並切換過去
<code>git checkout -</code>	切換回上一個 branch
<code>git checkout ^{SHA}<commit></code>	將 HEAD 移到指定 commit
<code>git checkout ^{SHA}<commit> <檔案></code>	某個檔案恢復成指定 commit 的版本 用於單一檔案回到先前狀況

題目 18：在新分支上工作

進入 `03-two-branches`，這裡已有 `main` 和 `ursidae` 兩個分支：

1. 在 `main` 分支相同位置建立一個 `fish` 分支
2. 在 `fish` 分支建立 `salmon.html` 檔案，並且 `commit` 第一次
3. 在相同分支對 `salmon.html` 檔案寫入一些內容，並且 `commit` 第二次
4. 回到 `main` 分支看一下 `salmon.html` 檔案現在長怎樣

✓ 03-TWO-BRANCHES

> .git

cat.html

dog.html

pig.html

Branches: Show All ▼

✓ Show Remote Branches

🔍 📄 ⚙️ ↺

Graph

Description

Date

Author

Commit

•	ursidae	chore: add panda	20 Mar 2026 11:47	krystal	47a948f6
•		chore: add bear	13 Mar 2026 11:41	krystal	6ef02a0e
○	fish	main chore: add dog	13 Mar 2026 11:40	krystal	a7561f2b
•		chore: add pig	13 Mar 2026 11:36	krystal	14990bc4
•		style: add text to the cat.html	13 Mar 2026 11:36	krystal	6c675884
•		chore: add cat	13 Mar 2026 11:33	krystal	fe1eb95c

問題

輸出

偵錯主控台

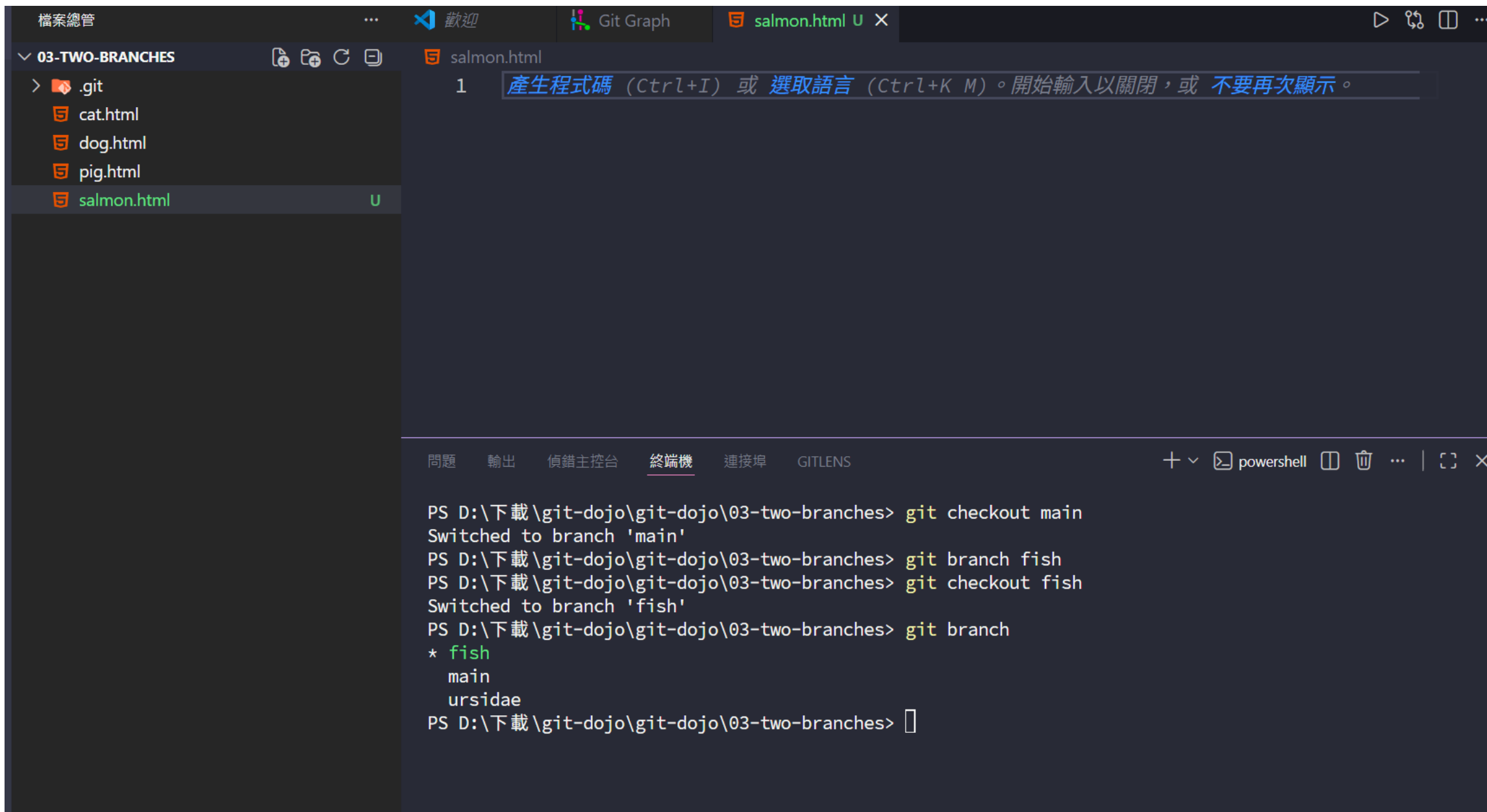
終端機

連接埠

GITLENS

+ ∨ 📄 powershell □ 🗑️ ... | 📄 ✕

```
PS D:\下載\git-dojo\git-dojo\03-two-branches> git checkout main
Switched to branch 'main'
PS D:\下載\git-dojo\git-dojo\03-two-branches> git branch fish
PS D:\下載\git-dojo\git-dojo\03-two-branches> git checkout fish
Switched to branch 'fish'
PS D:\下載\git-dojo\git-dojo\03-two-branches> git branch
* fish
  main
  ursidae
PS D:\下載\git-dojo\git-dojo\03-two-branches> 
```



檔案(F) 編輯(E) 選取項目(S) 檢視(V) ... 03-two-branches

檔案總管 ... 歡迎 Git Graph salmon.html x

03-TWO-BRANCHES .git cat.html dog.html pig.html salmon.html

salmon.html

You, 現在 | 1 author (You)

1 在相同分支對鮭魚檔案寫入一些內容，並且 commit 第二次

2

問題 輸出 偵錯主控台 終端機 連接埠 GITLENS powershell

```
PS D:\下載\git-dojo\git-dojo\03-two-branches> git checkout main
Switched to branch 'main'
PS D:\下載\git-dojo\git-dojo\03-two-branches> git branch fish
PS D:\下載\git-dojo\git-dojo\03-two-branches> git checkout fish
Switched to branch 'fish'
PS D:\下載\git-dojo\git-dojo\03-two-branches> git branch
* fish
  main
  ursidae
PS D:\下載\git-dojo\git-dojo\03-two-branches> git add salmon.html
PS D:\下載\git-dojo\git-dojo\03-two-branches> git commit -m '新增鮭魚檔案'
[fish fc7eafa] 新增鮭魚檔案
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 salmon.html
PS D:\下載\git-dojo\git-dojo\03-two-branches> git add salmon.html
PS D:\下載\git-dojo\git-dojo\03-two-branches> git commit -m '新增文字在鮭魚檔案裡'
[fish 2c35be8] 新增文字在鮭魚檔案裡
1 file changed, 1 insertion(+)
PS D:\下載\git-dojo\git-dojo\03-two-branches>
```

檔案總管

03-TWO-BRANCHES

> .git

cat.html

dog.html

pig.html

Branches: Show All

Show Remote Branches

Graph

Description

Date

Author

Commit

	新增文字在鮭魚檔案裡	27 Mar 2026 12:20	Molly	2c35be8f
	新增鮭魚檔案	27 Mar 2026 12:19	Molly	fc7eafa6
	chore: add panda	20 Mar 2026 11:47	krystal	47a948f6
	chore: add bear	13 Mar 2026 11:41	krystal	6ef02a0e
	chore: add dog	13 Mar 2026 11:40	krystal	a7561f2b
	chore: add pig	13 Mar 2026 11:36	krystal	14990bc4
	style: add text to the cat.html	13 Mar 2026 11:36	krystal	6c675884
	chore: add cat	13 Mar 2026 11:33	krystal	fe1eb95c

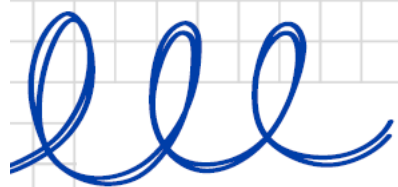
問題 輸出 偵錯主控台 終端機 連接埠 GITLENS

+ powershell

```
PS D:\下載\git-dojo\git-dojo\03-two-branches> git checkout main
Switched to branch 'main'
PS D:\下載\git-dojo\git-dojo\03-two-branches> git branch fish
PS D:\下載\git-dojo\git-dojo\03-two-branches> git checkout fish
Switched to branch 'fish'
PS D:\下載\git-dojo\git-dojo\03-two-branches> git branch
* fish
  main
  ursidae
PS D:\下載\git-dojo\git-dojo\03-two-branches> git add salmon.html
PS D:\下載\git-dojo\git-dojo\03-two-branches> git commit -m '新增鮭魚檔案'
[fish fc7eafa] 新增鮭魚檔案
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 salmon.html
PS D:\下載\git-dojo\git-dojo\03-two-branches> git add salmon.html
PS D:\下載\git-dojo\git-dojo\03-two-branches> git commit -m '新增文字在鮭魚檔案裡'
[fish 2c35be8] 新增文字在鮭魚檔案裡
1 file changed, 1 insertion(+)
PS D:\下載\git-dojo\git-dojo\03-two-branches> git checkout main
Switched to branch 'main'
PS D:\下載\git-dojo\git-dojo\03-two-branches>
```

> 大綱

> 時間表



git 跟 github 差異



- 一個版本控制工具（軟體）
- 安裝在自己的電腦上（本機）
- 用來管理檔案的歷史紀錄

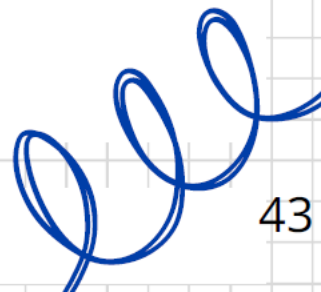
例如：Word



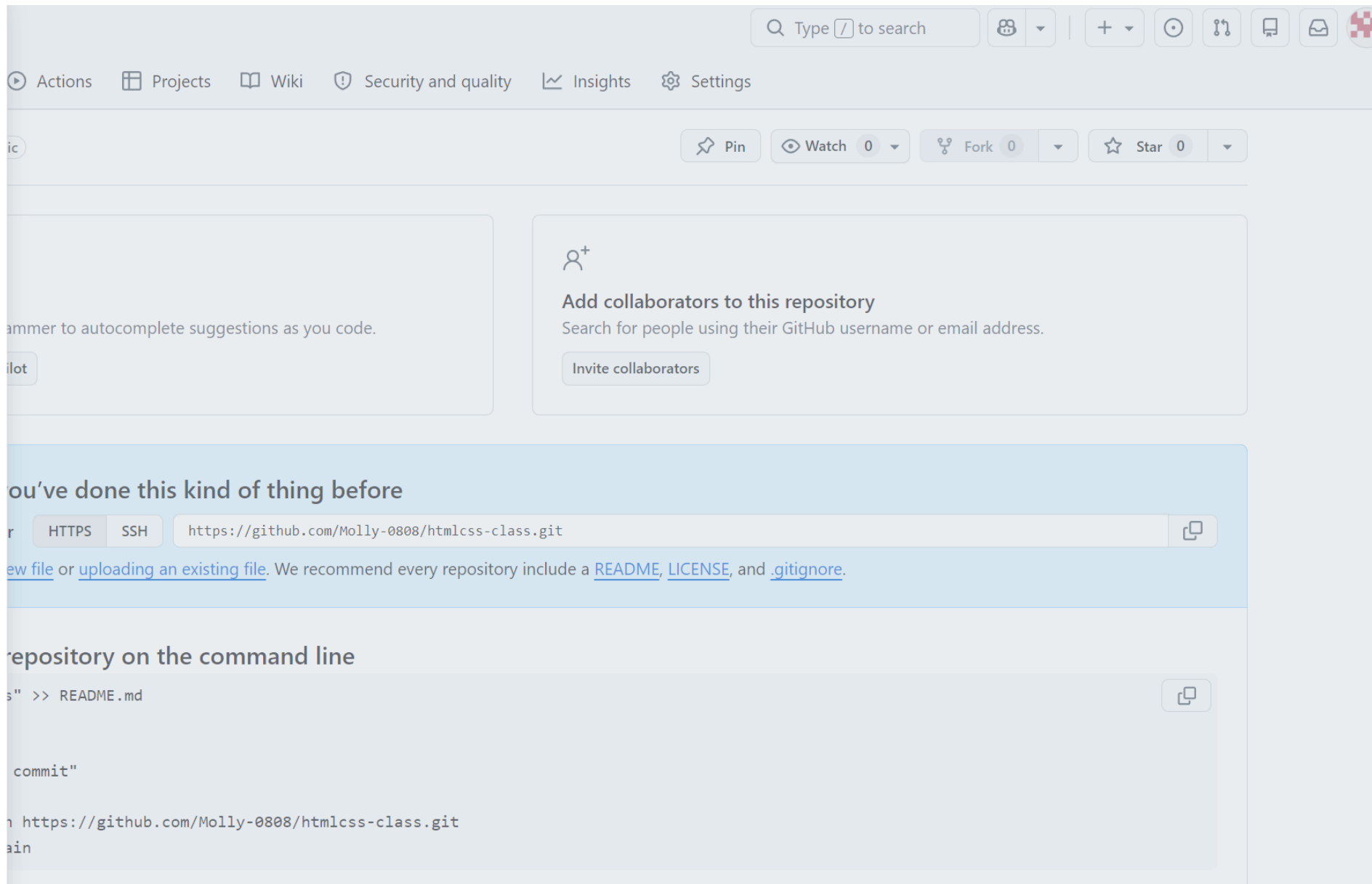
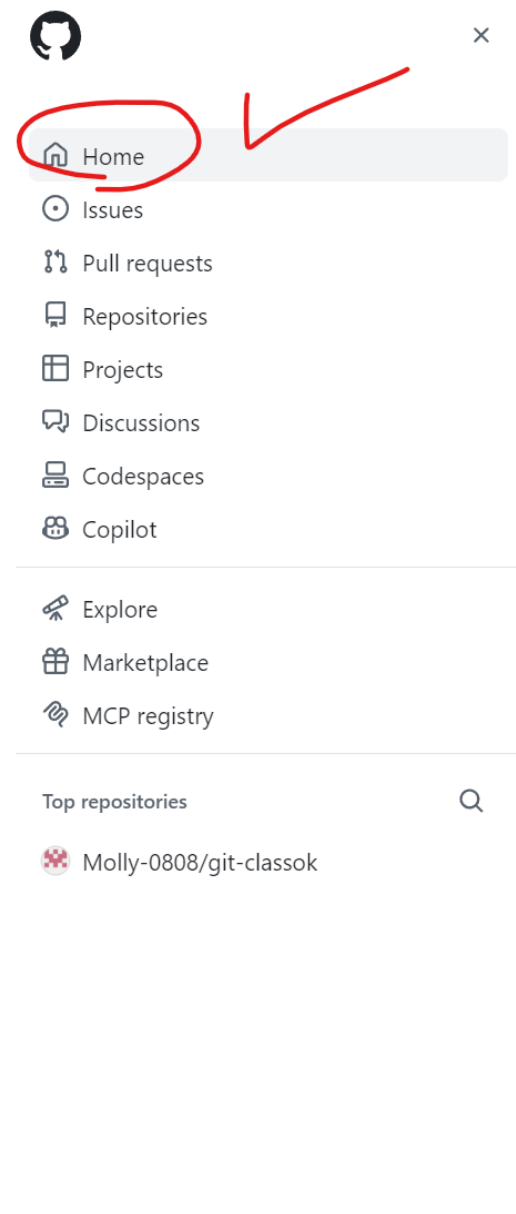
- 一個網站平台（網頁服務）
- 用來存放 Git 專案（遠端）
- 提供協作、分享、備份功能

例如：Google Drive

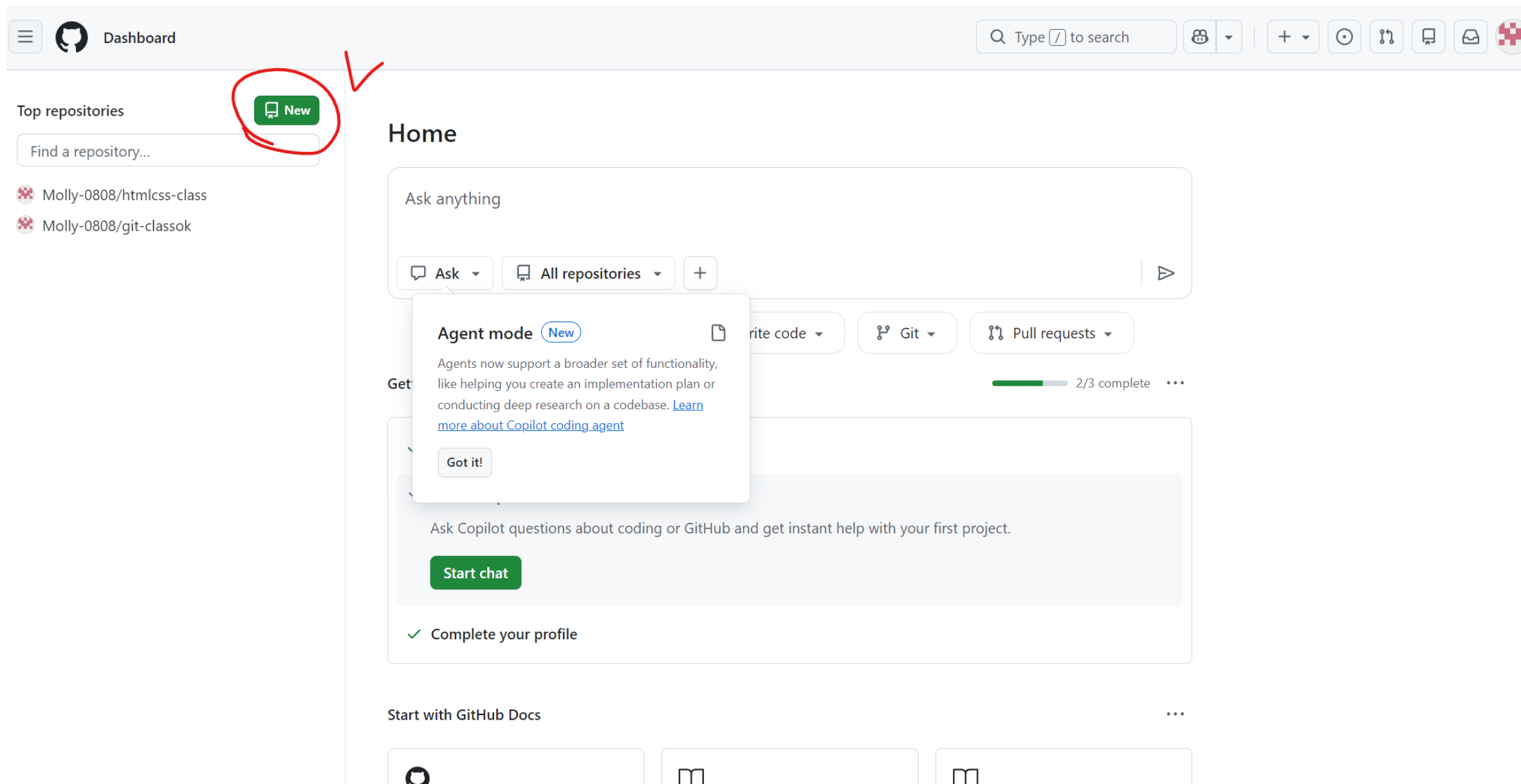
★ 我可以在本機用 Git 管理版本，並把專案放到 GitHub 上，方便分享與團隊協作



如何開新的github儲存區



如何開新的github儲存區



The screenshot shows the GitHub Dashboard interface. In the top-left corner, there is a 'Dashboard' header with a menu icon. Below it, the 'Top repositories' section lists two repositories: 'Molly-0808/htmlcss-class' and 'Molly-0808/git-classok'. A red circle highlights a green 'New' button with a repository icon, located to the right of the repository list. A red checkmark is placed above the 'New' button. The main content area is titled 'Home' and features a large 'Ask anything' section with a search bar and a 'Ask' button. Below this, there is a progress bar indicating '2/3 complete' and a 'Start chat' button. A notification box for 'Agent mode' is overlaid on the 'Ask anything' section, stating that agents now support a broader set of functionality and providing a link to 'Learn more about Copilot coding agent'. At the bottom, there is a 'Complete your profile' section with a green checkmark and a 'Start with GitHub Docs' section with a menu icon.

Dashboard

Top repositories

Find a repository...

Molly-0808/htmlcss-class

Molly-0808/git-classok

New

Home

Ask anything

Ask All repositories +

Agent mode **New**

Agents now support a broader set of functionality, like helping you create an implementation plan or conducting deep research on a codebase. [Learn more about Copilot coding agent](#)

Got it!

Write code Git Pull requests

2/3 complete

Start chat

✓ Complete your profile

Start with GitHub Docs

如何開新的github儲存區

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner *

 Molly-0808

Repository name *

Great repository names are short and memorable. How about [scaling-eureka](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

No license

比較適合當基本指令 - 1

git push 基本指令 - 2

指令	說明
<code>git remote add <遠端儲存庫暱稱> <遠端儲存庫網址></code>	在本地加入遠端 GitHub 專案，並取暱稱
<code>git push -u <遠端儲存庫暱稱> <本地分支></code>	推送本地分支到遠端，並且設定上游分支的對應 (--set-upstream) 之後可在 main 直接 git push、git pull

```
問題 輸出 偵錯主控台 終端機 連接埠 GITLENS + v powershell [警告圖示] [圖示] [圖示] ... | [圖示] [圖示] [圖示]
PS C:\Users\Galaxy\Desktop\git>
PS C:\Users\Galaxy\Desktop\git> git remote add class https://github.com/Molly-0808/git-class.git
PS C:\Users\Galaxy\Desktop\git> git push -u class main
fatal: User cancelled dialog.
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (7/7), 494 bytes | 494.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Molly-0808/git-class.git
 * [new branch]      main -> main
branch 'main' set up to track 'class/main'.
PS C:\Users\Galaxy\Desktop\git>
```

可自己換名字

如何連結github儲存區

步驟一：開vscode終端機

步驟二：git remote add origin <https://github.com/Molly-0808/htmlcss-class.git>(連結在github那邊)

步驟三：git branch -M main

(確認一下是master還是main，如果是master要記得寫
步驟三去改成main)

步驟四：git push -u origin main(首次推上去!!!)

```
$ git config --global init.defaultBranch main
```

效果：今後的master都自動變成main 不需要步驟三

git push 基本指令 - 1

儲存庫名預設origin
(可以換成自己想要的)

指令

git push

git push <儲存庫名> <分支名>

git push <儲存庫名> <本地分支>:
<遠端分支>

git push <儲存庫名> :<遠端分支>

git push <儲存庫名> <分支名> -f

說明

推送預設分支到預設的遠端儲存庫

推送分支到遠端儲存庫

本地的某分支推到遠端的某分支 (相同時
可以省略)

刪除遠端分支 (沒東西推上去 abc)

--force 縮寫，強制推送 (需小心使用)

.gitignore

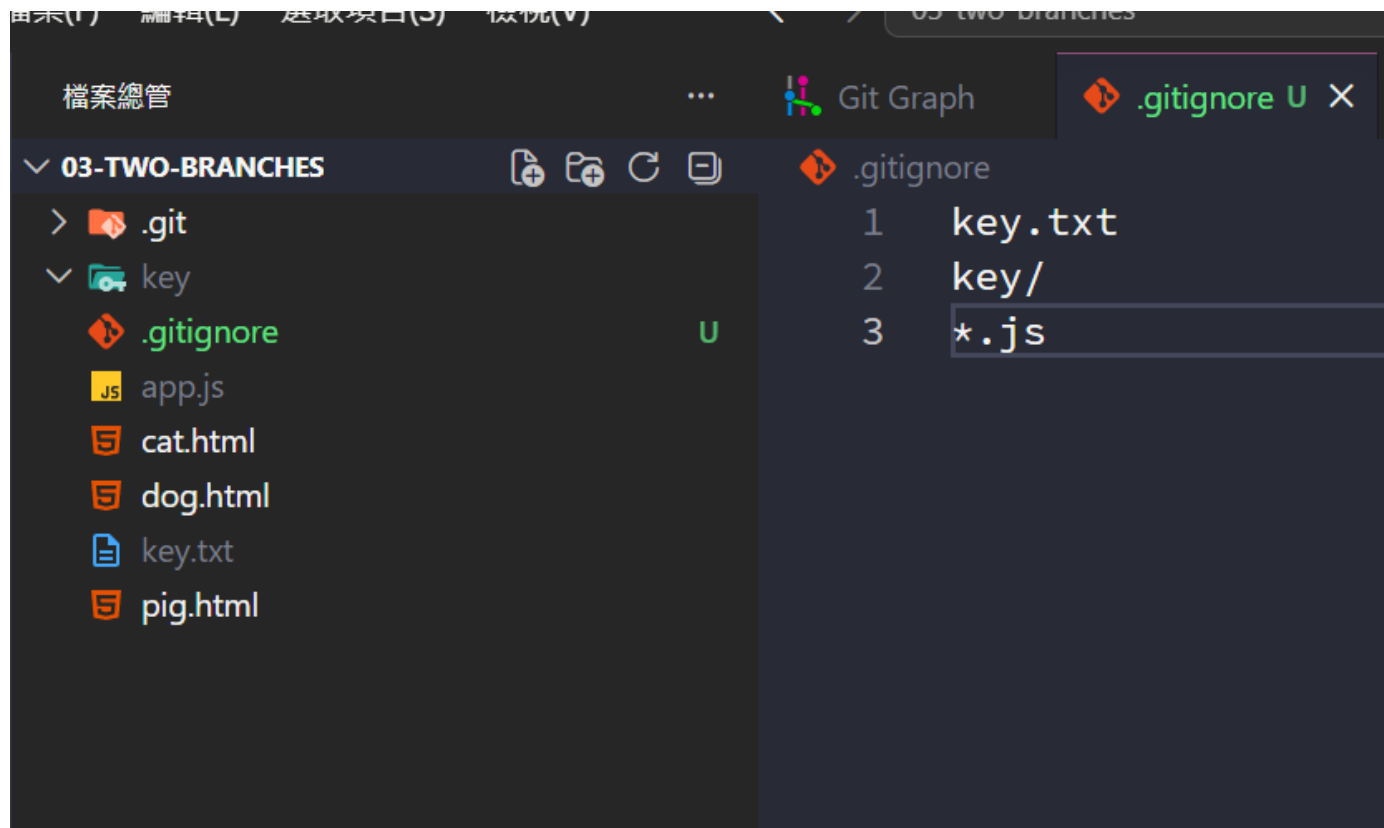
指定哪些檔案「不要被 Git 版控」

例如：金鑰（API KEY、密碼）、設定檔（.env）、編譯產物（node_modules、dist）、系統檔（.DS_Store）

Q: 已經存在版控的檔案我要怎 gitignore ？（既往不咎，先刪掉再重建）
`git rm --cached <檔名>`

空資料夾不會加進版控，他是用檔案內容去計算，可以放一個 .keep 檔案

```
◆ .gitignore
1  node_modules/  # node_modules 資料夾（含資料夾內所有檔案）
2  .env          # .env 檔案
3  *.log         # 所有副檔名為 .log 的檔案
```



在創建.gitignore之前已經被commit的檔案都不能被.gitignore作用 (因為不溯及)
首先 創立.gitignore 然後創立金鑰 (API KEY、密碼)、設定檔 (.env)、編譯產物
(node_modules、dist)、系統檔 (.DS_Store) 例如上方照片中的key.txt 和 key資料夾
然後把檔案的名字和資料夾名字加斜線「/」寫進.gitignore並儲存
該檔案和資料夾的顏色就會變成淺灰色

那我 API KEY 怎辦？

放進 .env 裡
是用來存放「不能公開的敏感資訊」的檔案

通常會有 .env.example 寫範例的金鑰

.env 不加版控，寫真的 API KEY
.env.example 加版控，不寫 KEY 的部分

「寫真的API KEY」意思就是把密碼們
都寫在這裡(.env)

```
# .env
API_KEY=sk-abc123xyz
```

```
# .env.example
API_KEY=
```

.env.example是給新加入專案
的人看到底有多少密碼種類，
再去跟建立專案的人要密碼